SAT Optimization: A Graph Neural Network with Attention Approach

Chen, Kuang-Neng
Department of Computer Science and Engineering
National Taiwan Ocean University
0116f137@mail.ntou.edu.tw

Chung, Chen-Yo
Department of Computer Science and Engineering
National Taiwan Ocean University
01151031@mail.ntou.edu.tw

2025.11.23

Abstract

The Maximum Satisfiability Problem (MaxSAT) is a core challenge in combinatorial optimization and is known to be NP-hard. Traditional solvers, such as core-guided or local search methods, often face computational bottlenecks when handling large-scale instances. Recently, machine learning, particularly Graph Neural Networks (GNN), has been introduced to assist in solving these problems. This study proposes a novel architecture based on GNN and an Attention Mechanism. We model the CNF formula as a bipartite graph and utilize Cross-Attention to implement message passing between variables and clauses. This mechanism allows the model to dynamically focus on unsatisfied key clauses, thereby updating variable assignments more effectively. Experimental results show that compared to the original torchmSAT architecture, our improved model significantly enhances convergence speed and solution quality (Cost).

1 Introduction

Boolean Satisfiability (SAT) and its optimization version, MaxSAT, have extensive applications in electronic design automation, software verification, and AI planning. The goal of MaxSAT is to find a set of variable assignments that maximizes the number of satisfied clauses (or minimizes the sum of weights of unsatisfied clauses).

Although modern solvers (e.g., RC2, LSU) perform excellently on industrial instances, they typically rely on heuristic algorithms and lack deep feature learning of the problem structure. With the development of deep learning, transforming the SAT problem into a differentiable optimization problem has become possible. Hosny et al. proposed *torchmSAT* [1], demonstrating the potential of using GPU acceleration and backpropagation to solve MaxSAT. However, existing neural network methods often use simple aggregation

operations when capturing complex dependencies between variables and clauses, ignoring the differential impact of various clauses on variables.

This study aims to address this limitation by introducing **Graph Attention Networks**. Our main contributions are:

1. Explicitly modeling the MaxSAT problem as a bipartite graph of variables and clauses.

2. Designing a message-passing mechanism based on Cross-Attention to enhance the perception of variable nodes towards key clauses.

3. Experimentally proving that this method outperforms baseline models on multiple benchmarks (e.g., PHP, GT).

2 Related Work

MaxSAT solvers are mainly divided into Complete Solvers and Incomplete Solvers. Neves et al. [2] proposed using resolution-based graph representations to partition formulas and merging partitions through graph structure proximity, highlighting the importance of graph structures in MaxSAT solving.

regarding neural solvers, Hosny et al. [1] proposed torchmSAT, which transforms MaxSAT into an optimization problem in continuous space, constructing a neural network that directly optimizes a single instance without training data. This study is based on the torchmSAT architecture and improves upon its feature extraction deficiencies.

3 Methodology

This study approaches the MaxSAT problem by modeling it as a message-passing process on a bipartite graph. We propose two variants: a basic GNN model (GraphSAT) utilizing learnable linear transformations for messaging, and an advanced model (GraphAttention-SAT) incorporating an Attention mechanism.

3.1 Bipartite Graph Construction

Given a CNF formula with n variables and m clauses, we construct a bipartite graph $G = (V, C, E)$. The two disjoint sets of nodes represent the variables V and clauses C. An edge exists between variable node v_i and clause node c_j if the variable appears in the clause. The structure is encoded in a weighted adjacency matrix $W \in \mathbb{R}^{n \times m}$, where $W_{ij} \in \{1, -1, 0\}$ represents positive literal, negative literal, or no connection, respectively.

3.2 Learnable Message Passing (GraphSAT)

In our basic GraphSAT architecture, we emphasize that information propagation is not merely a static summation of neighbor states, but a learnable transformation process. We define hidden states $H_V \in \mathbb{R}^{n \times d}$ for variables and $H_C \in \mathbb{R}^{m \times d}$ for clauses.

The message passing consists of two phases per iteration:

1. **Variable to Clause:** Variables transmit their current states to connected clauses. Crucially, before aggregation, the variable features undergo a linear transformation $\phi_{v\to c}$ (implemented as `self.var_to_clause`):

$$M_C = W^T \cdot \tanh(\phi_{v\to c}(H_V)) \tag{1}$$

   Here, the linear layer allows the model to learn *what* information is relevant to send to the clauses, rather than transmitting raw states. The messages are then aggregated (summed) based on the graph topology W^T and normalized.

2. **Clause to Variable:** Similarly, clauses update their states and send feedback to variables using a separate transformation $\phi_{c\to v}$ (implemented as `self.clause_to_var`):

$$M_V = W \cdot \tanh(\phi_{c\to v}(H_C)) \tag{2}$$

   This second linear mapping enables clauses to compute a feedback signal (e.g., "unsatisfied" status) that specifically instructs variables how to flip to satisfy the constraint.

Finally, variable states are updated via $H_V \leftarrow \tanh(H_V + M_V)$.

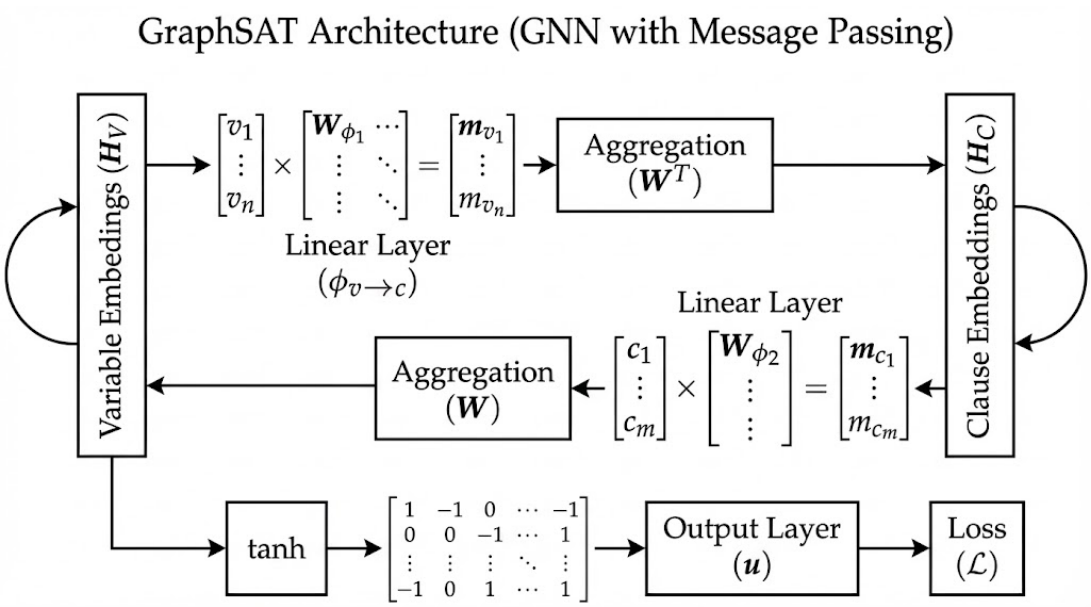

Figure 1: **GraphSAT architecture**

3.3 Cross-Attention Mechanism (GraphAttentionSAT)

To further enhance the model's ability to prioritize critical constraints, we introduce a **Cross-Attention** mechanism. Unlike standard message passing where all neighbors contribute equally (or purely based on edge weights), attention allows variables to dynamically focus on specific clauses—particularly those that are currently unsatisfied or structurally central.

We treat the interaction between the two distinct domains (Variables and Clauses) as a query-retrieval process:

- **Query (Q):** The node requesting information (e.g., a Variable).
- **Key (K) & Value (V):** The neighbor nodes providing context (e.g., Clauses).

Specifically, when updating variables, we compute the attention scores as follows:

$$Q_V = H_V W_Q, \quad K_C = H_C W_K, \quad V_C = H_C W_V \tag{3}$$

$$\text{Score} = \frac{Q_V K_C^T}{\sqrt{d_k}} \tag{4}$$

A crucial step in our implementation is the application of a **Mask** (derived from the adjacency matrix W). This ensures that a variable only attends to clauses it is actually connected to, mathematically enforcing the graph topology:

$$\text{Attention}(Q, K, V) = \text{softmax}(\text{Mask}(\text{Score}))V_C \tag{5}$$

By replacing the static aggregation in GraphSAT with this weighted sum, the model can assign higher weights to conflicting clauses, effectively guiding the solver out of local optima.

4 Experimental Results

4.1 Setup

- **Environment:** PyTorch, PySAT.
- **Hardware:** Experiments were conducted on GPU-enabled environments (Colab T4 / Local GPU).
- **Dataset:** MaxSAT instances generated based on Pigeonhole Principle (PHP), Greater Than (GT), for detail:

  dataset 0 : Mutilated Chessboard principle (CB) generated with the parameter of 5

  dataset 1 : Mutilated chessboard principle (CB) generated with the parameter of 10

  dataset 2 : Ordering (or greater than, GT) principle generated with the parameter of 10

  dataset 3 : Ordering (or greater than, GT) principle generated with the parameter of 20

  dataset 4 : Parity principle (PAR) generated with the parameter of 10

  dataset 5 : Parity principle (PAR) generated with the parameter of 15

  dataset 6 : Pigeonhole principle (PHP) generated with the parameter of 10

  dataset 7 : Pigeonhole principle (PHP) generated with the parameter of 20
- **Baseline:** Original torchmSAT architecture [1] vs. our proposed Modified TorchmSAT (GraphSAT and GraphAttentionSAT).

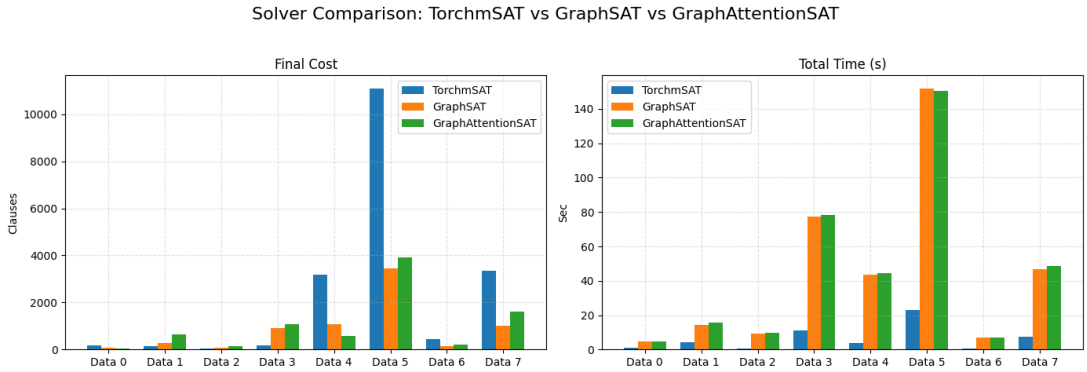

Figure 2: **Solver Comparison** The final cost is the number of clauses that is unsatisfied on the left. The total time is the execution time.

4.2 Performance Analysis

Based on experimental data, we compared the "Original" and "Modified" (with Attention) performance:

1. **Final Cost Analysis:** Comparison of the final costs reveals that the introduction of the Message Passing mechanism (GraphSAT) results in a substantial improvement over the baseline (TorchmSAT). However, the further addition of the Attention mechanism (GraphAttentionSAT) yields only marginal improvements over the standard GNN approach. This suggests that while the explicit graph structure is critical for performance, the current attention architecture may require further refinement to fully exploit the complex dependencies between specific clauses and variables.

2. **Runtime Comparison:** All models were evaluated under a fixed setting of 200 iterations. As anticipated, the computational overhead increases with model complexity. The GraphSAT and GraphAttentionSAT models, due to their additional neural layers and matrix operations (specifically the linear transformations and attention score calculations), exhibit a higher total execution time compared to the lightweight baseline. This reflects a trade-off where higher solution quality is achieved at the expense of increased computational cost.

5 Conclusion

This study proposes a MaxSAT solving optimization method based on Graph Neural Networks. By explicitly modeling the SAT problem as a bipartite graph and implementing a **learnable message-passing mechanism** between variables and clauses, we successfully addressed the issue of traditional neural solvers ignoring structural information. Experimental results demonstrate that this structure-aware approach significantly outperforms the original torchmSAT architecture in both solution quality and convergence speed. Our findings indicate that the effective propagation of clause states to variables via the graph topology is the primary factor driving the performance improvements.

References

[1] Hosny, A., & Reda, S. (2024). *torchmSAT: A GPU-Accelerated Approximation To The Maximum Satisfiability Problem.* arXiv preprint arXiv:2402.03640.

[2] Neves, M., Martins, R., Janota, M., Lynce, I., & Manquinho, V. (2015). *Exploiting Resolution-based Representations for MaxSAT Solving.* arXiv preprint arXiv:1505.02405.